

CS3231 - Theory of Computation
Audit
Summary

Sim Ray En Ryan

December 14, 2025

Contents

1	Sets and Regular Expressions	4
1.1	Basic Concepts and Definitions	4
1.1.1	Language and Alphabet	4
1.1.2	Set Operations on Languages	4
1.1.3	Conventions and Remarks	4
1.1.4	Regular Expressions and Languages	5
1.2	Language Growth	5
1.3	Properties of Set Operations	5
2	Grammars and the Chomsky Hierarchy	6
2.1	Grammars and Derivations	6
2.2	The Chomsky Hierarchy	7
2.3	Regular Languages: Equivalence of Notions	7
2.4	Pumping Lemmas	7
2.4.1	Traditional Pumping Lemma	7
3	Finite Automata	8
3.1	Deterministic Finite Automaton (DFA)	8
3.2	DFA and Regular Languages	8
3.3	Minimal DFA and Derivatives	8
4	Nondeterministic Finite Automata (NFA)	9
4.1	Definition and Equivalence to DFA	9
4.2	Summary of Regular Language Characterizations	9
5	Combining Languages	10
5.1	Closure Properties of Regular Languages	10
5.2	Closure Properties for Chomsky Hierarchy Levels	10
5.3	Homomorphisms	10
6	Normalforms and Algorithms	11
6.1	Normal Forms for Context-Free Grammars	11
6.2	Derivation Trees and Pumping	11
7	Deterministic Membership Testing (Parsing)	12
7.1	Context-Free Languages: CYK Algorithm	12
7.2	Linear Grammars	12
7.3	Context-Sensitive Languages: Savitch's Algorithm	12
8	Nondeterministic Membership Testing (Pushdown Automata)	13
8.1	Pushdown Automata (PDA)	13
8.2	Deterministic Context-Free Languages	13

9	Models of Computation	14
9.1	Turing Machines	14
9.2	Register Machines	14
9.3	Recursive Functions	14
10	Complexity Considerations	15
10.1	Complexity Classes	15
10.2	Undecidability	15
11	Undecidable Problems (Arithmetics and Diophantine Sets)	16
11.1	Diophantine Sets	16
11.2	Arithmetical Sets	16
12	Undecidability and Formal Languages	16
12.1	Multi Counter Automata and Equivalence to RE	16
12.2	Undecidable Problems in Formal Languages	17

1 Sets and Regular Expressions

1.1 Basic Concepts and Definitions

1.1.1 Language and Alphabet

- A language L is usually a set of strings w over an alphabet Σ .
- The alphabet Σ is usually finite.
- Concatenation of strings v and w is denoted by vw or $v \cdot w$.

1.1.2 Set Operations on Languages

Definition 1.1

- (a) **Finite list**: A finite list in set brackets denotes the set of the corresponding elements, e.g., $\{001, 0011, 00111\}$.
- (b) **Kleene Star (L^*)**: For any set L , L^* is the set of all strings obtained by concatenating finitely many strings from L :

$$L^* = \{u_1 \cdot u_2 \cdot \dots \cdot u_n : n \in \mathbb{N} \wedge u_1, u_2, \dots, u_n \in L\}$$

- (c) **Union ($L \cup H$)**: The set of all strings which are in L or in H .
- (d) **Intersection ($L \cap H$)**: The set of all strings which are in L and in H .
- (e) **Concatenation (LH)**: The set of concatenations of members of L and H : $\{v \cdot w : v \in L \wedge w \in H\}$.
- (f) **Set Difference ($L - H$)**: The set $\{u : u \in L \wedge u \notin H\}$.

1.1.3 Conventions and Remarks

Remarks 1.2

- The symbol $\emptyset$ denotes the empty set, also written as $\{\}$.
- The symbol ϵ denotes the empty string, and $\{\epsilon\}$ is the set containing the empty string.
- Σ^* is the set of all strings over the alphabet Σ .
- $\emptyset^* = \{\epsilon\}^* = \{\epsilon\}$.
- The operation $L \mapsto L^*$ is called the **Kleene star operation**.
- **Formal languages** are languages L for which there is a mechanism to check membership or to generate all members. Mechanisms include:

- **Automaton** (or machine): checks if a given word w belongs to L .
- **Grammar**: generates all words w belonging to L using step-wise rules.
- **Transducer**: a function that translates words such that L is the image of another language H .
- **Expression**: a short-hand description like a regular expression.

1.1.4 Regular Expressions and Languages

- **Regular languages** are those languages which can be defined using **regular expressions**.
- **Definition 1.3**: A regular expression denotes either a finite set (by listing elements), the empty set ($\emptyset$), or is formed from other regular expressions by the operations in Definition 1.1 (Kleene star, concatenation, union, intersection, set difference).
- **Convention**: L^+ denotes the set of all non-empty concatenations over members of L , i.e., $L^+ = L \cdot L^*$. Sometimes $L + H$ is used for $L \cup H$.

1.2 Language Growth

Definition 1.12

- A regular language L has **polynomial growth** iff there is a constant k such that at most n^k words in L are strictly shorter than n .
- A regular language L has **exponential growth** iff there are constants h, k such that, for all n , there are at least 2^n words shorter than $n \cdot k + h$ in L .

Theorem 1.13: Every regular language has either polynomial or exponential growth.

1.3 Properties of Set Operations

Theorem 1.9 (Lyndon and Schützenberger): If two words v, w satisfy $vw = wv$ then there is a word u such that $v, w \in u^*$. If a language L contains only words v, w with $vw = wv$ then $L \subseteq u^*$ for some u .

Theorem 1.10 (Structural Induction): Let P be a property of languages such that:

- Every finite set of words satisfies P ;
- If L, H satisfy P so do $L \cup H$, LH and L^* .

Then every regular set satisfies P .

Proposition 1.15 (Equality Rules): The following equality rules apply to any sets:

- (a) $L \cup L = L$, $L \cap L = L$, $(L^*)^* = L^*$, $(L^+)^+ = L^+$.

- (b) $(L \cup H)^* = (L^* \cdot H^*)^*$ and if $\epsilon \in L \cap H$ then $(L \cup H)^* = (L \cdot H)^*$.
- (c) $(L \cup \{\epsilon\})^* = L^*$, $\emptyset^* = \{\epsilon\}$ and $\{\epsilon\}^* = \{\epsilon\}$.
- (d) $L^+ = L \cdot L^* = L^* \cdot L$ and $L^* = L^+ \cup \{\epsilon\}$.
- (e) $(L \cup H) \cdot K = (L \cdot K) \cup (H \cdot K)$ and $K \cdot (L \cup H) = (K \cdot L) \cup (K \cdot H)$. (Distributivity of Concatenation over Union)
- (f) $(L \cup H) \cap K = (L \cap K) \cup (H \cap K)$ and $(L \cap H) \cup K = (L \cup K) \cap (H \cup K)$. (Distributivity of Intersection and Union)
- (g) $(L \cup H) - K = (L - K) \cup (H - K)$ and $(L \cap H) - K = (L - K) \cap (H - K)$.

Proposition 1.16 (Inequality Rules): The following inequality rules apply to any sets:

- (a) LL can be different from L .
- (b) $(L \cap H)^* \subseteq L^* \cap H^*$.
- (e) $(L \cap H) \cdot K \subseteq (L \cdot K) \cap (H \cdot K)$.
- (f) $K \cdot (L \cap H) \subseteq (K \cdot L) \cap (K \cdot H)$.

Corollary 1.17 (Characterization of Kleene Star and Plus): For any set L :

- (a) L^* is the smallest set H such that $\{\epsilon\} \cup (L \cdot H) = H$;
- (c) L^+ is the smallest set H such that $L \cup (L \cdot H) = H$.

2 Grammars and the Chomsky Hierarchy

2.1 Grammars and Derivations

- Grammars provide a recursive way to define languages.
- **Definition 2.2 (Grammar):** A grammar (N, Σ, P, S) consists of:
 - N : A finite set of **non-terminal** symbols.
 - Σ : A finite set of **terminal** symbols ($\Sigma \cap N = \emptyset$).
 - P : A set of **rules** (or productions), each of the form $l \rightarrow r$, where l is a string containing at least one symbol from N .
 - S : A **starting symbol**, $S \in N$.
- **Derivation:** w can be **derived from** v in one step ($v \Rightarrow w$) iff there are x, y and a rule $l \rightarrow r$ such that $v = xly$ and $w = xry$.
- $v \Rightarrow^* w$ means w can be derived from v in an arbitrary number of steps.
- The grammar generates the language $L = \{w \in \Sigma^* : S \Rightarrow^* w\}$.

2.2 The Chomsky Hierarchy

Definition 2.7: A grammar (N, Σ, P, S) belongs to the highest level of the following that applies:

- (CH3) **Regular (or right-linear):** Every rule is of the form $A \rightarrow wB$ or $A \rightarrow w$ where $A, B \in N$ and $w \in \Sigma^*$.
- (CH2) **Context-free:** Every rule is of the form $A \rightarrow w$ with $A \in N$ and $w \in (N \cup \Sigma)^*$.
- (CH1) **Context-sensitive:** Every rule is of the form $uAv \rightarrow uwv$ with $A \in N$, $u, v, w \in (N \cup \Sigma)^*$ and $w \neq \epsilon$. The rule $S \rightarrow \epsilon$ is allowed if S does not appear on any right side.
- (CH0) **Recursively enumerable:** The most general case, no restrictions beyond the basic definition.

Theorem 2.8 (Characterization of Context-Sensitive Languages):

- A language L not containing ϵ is context-sensitive iff it can be generated by a grammar satisfying that every rule $l \rightarrow r$ satisfies $|l| \leq |r|$.
- A language L containing ϵ is context-sensitive iff it can be generated by a grammar satisfying that $S \rightarrow \epsilon$ is a rule and any further rule $l \rightarrow r$ satisfies $|l| \leq |r| \wedge r \in (N \cup \Sigma - \{S\})^*$.

2.3 Regular Languages: Equivalence of Notions

Theorem 2.10: A language L is generated by a regular expression iff it is generated by a regular grammar.

2.4 Pumping Lemmas

2.4.1 Traditional Pumping Lemma

Theorem 2.15 (Pumping Lemma)

- (a) **Regular Languages:** If $L \subseteq \Sigma^*$ is an infinite regular language, there is a constant k such that for every $u \in L$ of length at least k there is a representation $x \cdot y \cdot z = u$ such that $|xy| \leq k$, $y \neq \epsilon$ and $xy^*z \subseteq L$.
- (b) **Context-Free Languages:** If $L \subseteq \Sigma^*$ is an infinite context-free language, there is a constant k such that for every $u \in L$ of length at least k there is a representation $vwxyz = u$ such that $|wxy| \leq k$, $w \neq \epsilon \vee y \neq \epsilon$ and $vw^lxy^lz \in L$ for all $l \in \mathbb{N}$.

Corollary 2.16 (Weaker Regular Pumping Condition): If L is an infinite regular language, there is a constant k such that for each word $w \in L$ with $|w| > k$, one can represent w as $xyz = w$ with $y \neq \epsilon$ and $xy^*z \subseteq L$.

3 Finite Automata

3.1 Deterministic Finite Automaton (DFA)

- A DFA is a mechanism that checks membership in a language using a finite amount of memory (states).
- **Description 3.3:** A DFA is formally described as a tuple $(Q, \Sigma, \delta, s, F)$:
 - Q : The finite set of **states**.
 - Σ : The finite **alphabet** used.
 - δ : The **transition function** $\delta : Q \times \Sigma \rightarrow Q$.
 - s : The **starting state**, $s \in Q$.
 - F : The set of **accepting states**, $F \subseteq Q$.
- The transition function extends to strings: $\delta(q, \epsilon) = q$ and $\delta(q, wa) = \delta(\delta(q, w), a)$.
- The automaton **accepts** w iff $\delta(s, w) \in F$.

3.2 DFA and Regular Languages

Theorem 3.8 (DFA $\rightarrow$ Regular Language): If a language L is recognised by a deterministic finite automaton then L is regular.

- **Proof Idea:** Construct a regular grammar (Q, Σ, P, s) where the states Q are the non-terminals.
 - Rule $q \rightarrow ar$ is in P iff $\delta(q, a) = r$.
 - Rule $q \rightarrow \epsilon$ is in P iff $q \in F$.

Theorem 3.9 (Block Pumping Lemma for Regular Sets): If L is a regular set then there is a constant k such that for all strings $u_0, u_1, \dots, u_k$ with $u_0 u_1 \dots u_k \in L$ and $u_1, \dots, u_{k-1}$ being nonempty there are i, j with $0 < i < j \leq k$ and

$$(u_0 u_1 \dots u_{i-1}) \cdot (u_i u_{i+1} \dots u_{j-1})^* \cdot (u_j u_{j+1} \dots u_k) \subseteq L$$

Theorem 3.11 (Ehrenfeucht, Parikh and Rozenberg): A language L is regular if and only if both L and its complement satisfy the block pumping lemma.

3.3 Minimal DFA and Derivatives

Theorem 3.17 (Myhill and Nerode's Minimal DFA): Given a language L , let $L_x = \{y \in \Sigma^* : xy \in L\}$ be the **derivative** of L to x .

- The language L is regular iff the number of different derivatives L_x is finite.

- For languages with exactly n derivatives, one can construct a complete DFA having n states, and there is no complete DFA with fewer than n states that recognises L .

Lemma 3.21 (Jaffe's Matching Pumping Lemma): A language $L \subseteq \Sigma^*$ is regular iff there is a constant k such that for all $x \in \Sigma^*$ and $y \in \Sigma^k$ there are u, v, w with $y = uvw$ and $v \neq \epsilon$ such that, for all $h \in \mathbb{N}$, $L_{xuv^h w} = L_{xy}$.

Algorithm 3.29 (Myhill's and Nerode's Algorithm to Minimise DFA): A procedure to transform a complete DFA into an equivalent minimal complete DFA by identifying reachable states and states that are **distinct** (behave differently when starting from them).

4 Nondeterministic Finite Automata (NFA)

4.1 Definition and Equivalence to DFA

- **Description 4.2 (Nondeterministic Finite Automaton):** An NFA is a tuple $(Q, \Sigma, \delta, s, F)$ that differs from a DFA in that the transition function δ is multi-valued, where $\delta(q, a)$ is a **set** of states.
- A word w is **accepted** by an NFA iff there is at least one run on the input w whose last state is accepting.
- NFAs can be exponentially smaller than corresponding DFAs.
- **Theorem 4.5 (Determinisation of NFAs - Subset Construction):** For each NFA $(Q, \Sigma, \delta, s, F)$ with $n = |Q|$ states, there is an equivalent DFA whose 2^n states are the subsets $Q' \subseteq Q$.
 - States: $Pow(Q)$.
 - Start State: $\{s\}$.
 - Transition: $\delta'(Q', a) = \{q'' \in Q : \exists q' \in Q' [q'' \in \delta(q', a)]\}$.
 - Accepting States: $F' = \{Q' \subseteq Q : Q' \cap F \neq \emptyset\}$.
- **Theorem 4.13:** Every language generated by a regular grammar is also recognised by an NFA.

4.2 Summary of Regular Language Characterizations

Corollary 4.17: A language L is regular iff it satisfies any of the following equivalent conditions:

- L is generated by a regular expression.
- L is generated by a regular grammar.
- L is recognised by a deterministic finite automaton (DFA).

- (d) L is recognised by a nondeterministic finite automaton (NFA).
- (e) L and its complement satisfy both the block pumping lemma.
- (f) L satisfies Jaffe's pumping lemma.
- (g) L has only finitely many derivatives (Theorem of Myhill and Nerode).

5 Combining Languages

5.1 Closure Properties of Regular Languages

Description 5.2 (Product Automaton for Closure): Given two DFAs recognizing L_1 and L_2 , one can construct a **product automaton** $(Q_1 \times Q_2, \Sigma, \delta_1 \times \delta_2, (s_1, s_2), F)$ where F is chosen based on the operation:

- **Union** $(L_1 \cup L_2)$: $F = F_1 \times Q_2 \cup Q_1 \times F_2$.
- **Intersection** $(L_1 \cap L_2)$: $F = F_1 \times F_2$.
- **Difference** $(L_1 - L_2)$: $F = F_1 \times (Q_2 - F_2)$.

For concatenation and Kleene star, one typically constructs an NFA and then converts it to a DFA.

5.2 Closure Properties for Chomsky Hierarchy Levels

Theorem 5.1 (Basic Closure Properties): If L, H are languages on the level CHk of the Chomsky hierarchy, then $L \cup H$, LH and L^* are also on level CHk .

Theorem 5.9 (Context-Free and Regular Intersection): Assume that L is a context-free language and H is a regular language. Then the intersection $L \cap H$ is also a context-free language.

Theorem 5.12 (Context-Sensitive Concatenation): The concatenation of two context-sensitive languages is context-sensitive.

Theorem 5.13 (Context-Sensitive Kleene Star): If L is context-sensitive so is L^* .

Theorem 5.15 (Context-Sensitive Intersection): The intersection of two context-sensitive languages is context-sensitive.

5.3 Homomorphisms

Definition 5.28 (Homomorphism): A homomorphism is a mapping h from words to words satisfying $h(xy) = h(x) \cdot h(y)$ for all words x, y . The values $h(a)$ for the characters a define the image $h(w)$ of every word w .

Theorem 5.32 (Homomorphic Image Closure): The homomorphic image of regular and context-free languages are regular and context-free, respectively.

Theorem 5.47 (Inverse Homomorphism Closure): If L is on the level k of the Chomsky hierarchy and h is a homomorphism then $h^{-1}(L)$ is also on the level k of the Chomsky hierarchy.

6 Normalforms and Algorithms

6.1 Normal Forms for Context-Free Grammars

Definition 6.1 (Normalforms): For a context-free grammar (N, Σ, P, S) , assuming $S \rightarrow \epsilon$ if ϵ is generated, and no other rule $A \rightarrow \epsilon$ for $A \neq S$:

- **Chomsky Normal Form (CNF):** Every rule (except perhaps $S \rightarrow \epsilon$) is either of the form $A \rightarrow a$ or of the form $A \rightarrow BC$ for some $A, B, C \in N$ and terminal $a \in \Sigma$.
- **Greibach Normal Form (GNF):** Every rule (except perhaps $S \rightarrow \epsilon$) has a right hand side from ΣN^* .

Algorithm 6.2 (Chomsky Normal Form): An algorithm to convert any context-free grammar to CNF in four main steps: dealing with ϵ rules, dealing with single terminal letters, breaking long right hand sides, and removing unit rules ($A \rightarrow B$).

Algorithm 6.7 (Removal of Useless Nonterminals): A two-step algorithm to remove non-terminals that are either non-terminating (cannot derive a word of terminals) or unreachable (cannot be reached from the start symbol S).

Algorithm 6.11 (Emptiness Check): Checks if a context-free grammar generates any word by finding all non-terminals that can derive a terminal string.

Algorithm 6.12 (Finiteness Check): Checks if a context-free grammar generates an infinite set by looking for cycles in the dependencies of productive non-terminals.

6.2 Derivation Trees and Pumping

Description 6.18 (Derivation Tree): A representation of a derivation where nodes are labeled with non-terminals, leaves contain terminals, and a node with its successors corresponds to a rule in the grammar.

Theorem 2.15 (b) [Proof using Derivation Trees]: The existence of a repeated non-terminal along a path in a derivation tree for a long word guarantees the existence of the pump wxy with the required properties.

Theorem 6.22 (Ogden's Pumping Lemma): A stronger version of the context-free pumping lemma that allows marking symbols in the word to control where the pump occurs.

7 Deterministic Membership Testing (Parsing)

7.1 Context-Free Languages: CYK Algorithm

Algorithm 7.1 (Cocke, Kasami and Younger's Parser - CYK): An $O(n^3 \cdot m)$ algorithm for checking membership of a word $w = a_1 \dots a_n$ in a grammar in CNF.

- Variables $E_{i,j}$ store the set of non-terminals that can generate the subword $a_i \dots a_j$.
- **Initialisation:** $E_{k,k} = \{A \in N : A \rightarrow a_k \text{ is a rule}\}$.
- **Loop:** $E_{i,j} = \{A : \exists \text{ rule } A \rightarrow BC, \exists k \text{ with } i \leq k < j \text{ s.t. } B \in E_{i,k} \text{ and } C \in E_{k+1,j}\}$.
- **Decision:** w is generated iff $S \in E_{1,n}$.

7.2 Linear Grammars

Description 7.6 (Linear Grammars): A grammar where each rule has at most one non-terminal, e.g., $A \rightarrow u$ or $A \rightarrow vBw$.

Algorithm 7.7 (Parsing of Linear Grammars): An $O(n^2 \cdot m)$ algorithm for checking membership in a linear grammar in a normal form where rules are $A \rightarrow c$, $A \rightarrow cB$ or $A \rightarrow Bc$.

Algorithm 7.12 (Kleene Star of Linear Grammar): An $O(n^2)$ algorithm to check membership in L^* , where L is a linear language.

7.3 Context-Sensitive Languages: Savitch's Algorithm

Algorithm 7.21 (Savitch's Algorithm - Recursive Check): An algorithm that checks membership in a context-sensitive language L in space $O(n^2)$ by recursively dividing the derivation time in half.

Definition 7.26 (Growing Context-Sensitive Grammars): A grammar (N, Σ, P, S) is growing iff $|l| < |r|$ for all rules $l \rightarrow r$.

Theorem 7.27 (Growing CTS Membership is in P): Membership for languages generated by growing context-sensitive grammars is decidable in polynomial time (with respect to the word length).

8 Nondeterministic Membership Testing (Pushdown Automata)

8.1 Pushdown Automata (PDA)

Definition 8.2 (Pushdown Automaton): A PDA is a tuple $(Q, \Sigma, N, \delta, s, S, F)$ where:

- Q, Σ, s, F are the states, alphabet, start state, and final states, respectively.
- N is the set of **stack symbols** with S as the start symbol (on the stack).
- δ : The transition function, where a rule is a triple $(p, v, A) \mapsto (q, w)$, meaning in state p , reading input v and having A on top of the stack, the machine transitions to state q , consumes v , pops A , and pushes w ($w \in N^*$) onto the stack.
- **Acceptance by state:** Accepted if the input is processed and the machine is in an accepting state $q \in F$.
- **Acceptance by empty stack:** Accepted if the input is processed and the stack is empty.

Algorithm 8.4 (Grammar $\rightarrow$ PDA (Empty Stack)): Construction of a PDA accepting by empty stack from a CFG in CNF. The single state s has transitions $\delta(s, \epsilon, A) = \{(s, BC)\}$ for $A \rightarrow BC$ and $\delta(s, a, A) = \{(s, \epsilon)\}$ for $A \rightarrow a$.

Theorem 8.12: If L is recognized by a PDA accepting by final state, it can also be recognized by a PDA accepting by empty stack.

Algorithm 8.13 (PDA $\rightarrow$ Grammar): Given a PDA accepting by empty stack, one can construct an equivalent context-free grammar.

8.2 Deterministic Context-Free Languages

Definition 8.19 (Deterministic Pushdown Automaton): A PDA with acceptance by state such that for every combination of current state, stack top, and potential input prefix, there is at most one possible next move.

Proposition 8.20 (Closure under Complement): Deterministic context-free languages are closed under complement.

Proposition 8.21 (Closure under Intersection/Union with Regular): If L is deterministic context-free and H is regular, then $L \cap H$ and $L \cup H$ are deterministic context-free.

Theorem 8.23 (Non-Closure under Union/Intersection): The deterministic context-free languages are neither closed under union nor under intersection.

9 Models of Computation

9.1 Turing Machines

Definition 9.1 (Turing Machine): A model of computation with an infinite tape, a tape head, and a finite control (states).

- A TM is a tuple $(Q, \Gamma, \sqcup, \Sigma, \delta, s, F)$ where:
 - Q : set of states.
 - Γ : tape alphabet.
 - $\sqcup$: blanc symbol ($\sqcup \in \Gamma - \Sigma$).
 - Σ : input alphabet ($\Sigma \subset \Gamma$).
 - δ : transition function: $(Q - F) \times \Gamma \rightarrow Q \times \Gamma \times \{\text{left}, \text{right}\}$.
 - s : start state.
 - F : set of halting states.
- TMs compute partial functions.

9.2 Register Machines

Description 9.5 (Register Machine): A model of computation operating on natural numbers stored in a finite number of registers $R_1, R_2, \dots, R_n$, executing instructions like arithmetic, comparison, and jumps (Goto).

Theorem 9.12 (TM $\rightarrow$ Register Machine): Every Turing machine can be simulated by a register machine, and there is even a single universal register machine.

Theorem 9.15 (Equivalence of TM and Register Machines): The partial functions from $\Sigma^* \rightarrow \Sigma^*$ computed by a TM are the same as the partial functions from $N \rightarrow N$ computed by a register machine.

9.3 Recursive Functions

Definition 9.16 (Primitive Recursive Functions): Defined structurally:

- **Base Functions:** Constant (zero), Successor ($x \mapsto x + 1$), Projection ($(x_1, \dots, x_n) \mapsto x_m$).
- **Inductive Steps:** Composition, Recursion.

Primitive recursive functions are always total.

Definition 9.21 (Partial Recursive Functions): Formed from base functions by composition, primitive recursion, and μ -**minimalisation** ($g(\vec{x}) = \mu y[f(y, \vec{x}) = 0]$).

Theorem 9.23 (Equivalence of Models): For a partial function f , the following are equivalent:

- f is computable by a Turing machine.
- f is computable by a register machine.
- f is partial recursive.

Thesis 9.24 (Church's Thesis): All sufficiently reasonable models of computation are equivalent.

10 Complexity Considerations

10.1 Complexity Classes

Description 10.4 (The Complexity Class P): The class of sets decidable by an algorithm in **polynomial time** on input size n . Robust across different machine models (TM, register machine with restricted operations).

Description 10.6 (The Complexity Class NP): The class of sets decidable by a **non-deterministic** algorithm in polynomial time.

Description 10.15 (Complexity Class PSPACE): The class of sets decidable by an algorithm using **polynomial space** on input size n . PSPACE is conjectured to be strictly larger than P and NP.

Description 10.23 (Decidable/Recursive Sets):

- **Decidable (Recursive):** A set $L \subseteq N$ where the characteristic function is computable by a recursive function (e.g., a TM that always halts).
- **Recursively Enumerable (RE):** A set that is the range of a partial recursive function (e.g., recognizable by a TM that halts and accepts, but may run forever and reject).

Theorem 10.29: A set L is recursive iff both L and $N - L$ are recursively enumerable.

10.2 Undecidability

Definition 10.27 (Halting Problem):

- **General Halting Problem (H):** $H = \{(e, x) : \phi_e(x) \text{ is defined}\}$.
- **Diagonal Halting Problem (K):** $K = \{e : \phi_e(e) \text{ is defined}\}$.

Theorem 10.28 (Undecidability of the Halting Problem): Both H and K are recursively enumerable and undecidable.

Definition 10.25 (Recursively Enumerable): A set is recursively enumerable iff it is the range of a partial recursive function.

Theorem 10.25 (Equivalence of RE Notions): The notions of a set being recursively enumerable (range of partial recursive function) are equivalent to being the range of a total recursive or primitive recursive function, the domain of a partial recursive function, or the set of inputs on which some register machine terminates.

Observation 11.21 (Many-One Reduction): If A is many-one reducible to B (via a recursive function g such that $x \in A \Leftrightarrow g(x) \in B$) and B is recursively enumerable, then A is also recursively enumerable.

11 Undecidable Problems (Arithmetics and Diophantine Sets)

11.1 Diophantine Sets

Definition 11.1 (Diophantine Sets): A set $A \subseteq \mathbb{N}$ is Diophantine iff it can be expressed as the set of values x such that there exist natural numbers $y_1, \dots, y_n$ satisfying an equation $p(x, y_1, \dots, y_n) = q(x, y_1, \dots, y_n)$ for polynomials p, q with natural coefficients (or $p(x, y_1, \dots, y_n) = 0$ for a polynomial p with integer coefficients).

Proposition 11.7: Every Diophantine set is recursively enumerable.

Theorem 11.10 (Unsolvability of Hilbert's Tenth Problem): Every recursively enumerable set is Diophantine; in particular, there are undecidable Diophantine sets.

11.2 Arithmetical Sets

Definition 11.11 (Arithmetical Sets): A set $A \subseteq \mathbb{N}$ is arithmetical iff it can be defined by a formula using existential ($\exists$) and universal ($\forall$) quantifiers over variables such that the predicate behind the quantifiers only uses Boolean combinations of polynomials compared by $<$ and $=$.

Theorem 11.15 (Undecidability of Arithmetics): The set of all true formulas in arithmetic of the natural numbers with $+$ and $\cdot$ using universal ($\forall$) and existential ($\exists$) quantification over variables is undecidable.

12 Undecidability and Formal Languages

12.1 Multi Counter Automata and Equivalence to RE

Description 12.1 (Multi Counter Automata): A generalization of a PDA where the stack is replaced by a fixed number of counters (registers) that can only perform $+1$, -1 ,

and comparison to 0.

Theorem 12.2 (Register Machine $\rightarrow$ Counter Machine): Register machines can be translated into counter machines.

Corollary 12.6: Every language recognized by a Turing machine or a register machine can also be recognized by a counter machine.

Theorem 12.7 (Counter Machine $\rightarrow$ Grammar): If K is recognized by a counter machine then there are deterministic context-free languages L and H and a homomorphism h such that $K = h(L \cap H)$, implying K is generated by some grammar.

Theorem 12.10 (Equivalence of Grammar and RE): A set $K \subseteq \Sigma^*$ is recursively enumerable iff it is generated by some grammar. In particular, there are grammars for which it is undecidable which words they generate.

12.2 Undecidable Problems in Formal Languages

Corollary 12.11: The following questions are undecidable:

- Given a grammar and a word, does this grammar generate the word?
- Given two deterministic context-free languages by deterministic pushdown automata, does their intersection contain a word?
- Given a context-free language given by a grammar, does this grammar generate $\{0, 1, 2, 3, 4\}^*$?
- Given a context-sensitive grammar, does its language contain any word?

Description 12.12 (Post's Correspondence Problem (PCP)): An instance is a list of pairs of words $(x_1, y_1), \dots, (x_n, y_n)$. It has a solution if there is a sequence of indices $k_1, \dots, k_m$ such that $x_{k_1}x_{k_2}\dots x_{k_m} = y_{k_1}y_{k_2}\dots y_{k_m}$. PCP is undecidable.